Choosing a Fast Stand-In for a Slow Simulation: a Selection Tool and a 44-Problem Study

[Author names to be supplied by the authors][a]

[a] *[Affiliations and corresponding-author e-mail to be supplied by the authors]*

Abstract

Engineers who depend on a slow simulation can afford only a limited number of runs, and must decide which fast stand-in model to train and whether more runs are worth the cost. We present SurroSelect, an open tool that chooses a stand-in from a pool of classical models by cross-validation, reports which models are tied, checks whether the POD basis limits accuracy, and forecasts the error for more runs. We tested it against criteria committed before the evaluation runs, first on eight problems solved by the third-party finite-element library `scikit-fem` and then, with the tool frozen, on eight further ones; we also compared it with a tuned neural network and ran it on two real measured datasets, and a 44-problem study describes where each model family wins. On the further problems selection was within a median factor 1.04 of the oracle (the best pool member by test error), and always choosing one fixed pool member does almost as well. The tool was again worse than the benchmark's own selection among methods that include networks, in particular on wave problems, and the error forecast for four times the budget was within a factor two in only 68% of 96 cases over all sixteen problems, usually too optimistic. Adding a tuned network to the pool helped on one of the two wave problems, at several times the cost. For a solver taking 1.52 seconds per run, the surrogate was cheaper than the solver after about 335 predictions. All problems are simulated and their solvers are fast; the evidence for much slower simulations is missing.

Keywords: surrogate model, Gaussian process, neural network, reduced-order model, parametric PDE, model selection, learning curve

1. Introduction

Many engineering decisions rest on a simulation that is too slow to run as often as one would like. Choosing the shape of a part, finding how uncertain

an input leaves the answer, or fitting a model to measurements can each need thousands of runs. A common remedy is a *surrogate*, or stand-in model: the simulation is run a limited number of times, say a few hundred, a cheap model is trained on the inputs and outputs, and the cheap model then replaces the simulation in all further work [1]. For example, if one run takes an hour, five hundred runs take about three weeks on one machine, whereas a trained stand-in answers in milliseconds.

The practical question is which kind of stand-in to build, and how many runs are enough. One long-standing route compresses the output fields with proper orthogonal decomposition (POD) and regresses the weights of the leading modes on the inputs, with a neural network [2, 3] or a Gaussian process [4]. The other route trains a neural network to produce the whole field [5, 6]. Published comparisons are mixed. A survey of machine-learning solvers for fluid problems found that most papers claiming to beat a standard method used a weak baseline [7]; kernel and Gaussian-process methods have been reported to be competitive with DeepONet [8] and the Fourier neural operator [9] on benchmark problems [10]; and no universal winner was found between neural and polynomial surrogates [11]. A practitioner holding a few hundred runs of their own simulation still has no procedure that tells them which stand-in to use on *their* data, how far it can be trusted, or how many more runs it would take to reach a target accuracy.

Gap. Existing comparisons rank methods on fixed problem sets and do not give the practitioner a validated procedure to select a stand-in, to see whether the POD basis limits accuracy, and to forecast the number of runs needed.

Motivation and contributions

1. **A selection tool, SurroSelect** (section 4). It fits a pool of stand-in models, chooses by cross-validation while reporting which models are tied with the best, checks whether the POD basis is the bottleneck, and forecasts the error for more runs from a learning curve. It is free software with a command-line interface (Sec. S8).
2. **A pre-specified evaluation on problems solved by third-party code** (sections 6.3 and 6.7). Criteria were committed to version control before the evaluation runs on eight problems solved by the third-party finite-element library `scikit-fem` (smoke tests of the tool on three of them preceded the commit; see section 7, limitation 2). The tool was then frozen and the procedure repeated, with stated predictions, on eight further problems; we also test a tuned network in the pool, a larger tuning budget for the network, and a solver that takes seconds

per run. We report which criteria were met and which were not, against a neural network tuned with a larger budget than any other method, and on two real measured datasets.

3. **A 44-problem study of where each family wins** (sections 6.1 and 6.2). Controls separate the effects of the reduced basis, of the regressor, and of how widely its hyper-parameters are searched, and the outcome of a comparison is shown to depend on the training budget and the tuning effort.
4. **A record of what did not work** (section 7), including a pilot-data diagnostic that was not confirmed, a control that had to be widened after review, and every deviation from the plan (Supplement, Sec. S4).

The paper proposes no new regression model; the pool consists of classical methods, and the contribution is the procedure, its validation, and the evidence about when each member wins.

Organisation

Section 2 reviews related work; section 3 fixes notation and assumptions; section 4 describes the method; section 5 the experimental framework; section 6 the results; section 7 the discussion and limitations; section 8 concludes.

2. Related Work

2.1. Reduced-basis surrogates with learned coefficients

Non-intrusive reduced-order models regress the weights of POD modes on the parameters. Hesthaven and Ubbiali used neural networks for this [2], Bhattacharya et al. analysed PCA-based neural operators [3], and Guo and Hesthaven used Gaussian-process regression for nonlinear structural analysis [4]. Bērziņš et al. compared radial-basis-function, Gaussian-process and neural-network regression inside one POD framework for extruder flows and reported several advantages of the Gaussian process [12]; we read only the abstract, and it is the closest precedent for our finding on smooth problems. Mora et al. proposed a hybrid Gaussian-process/neural operator framework [13].

2.2. Neural networks, operators and nonlinear manifolds

DeepONet [8], the Fourier neural operator [9] and the neural-operator framework [14] learn maps between function spaces; autoencoders learn nonlinear latent manifolds for reduced models [15, 5, 6]. Our parameters

are finite-dimensional, so we study direct and bottleneck networks rather than operator-input architectures. Whether a small linear basis can work depends on the decay of the Kolmogorov n-width [16], which is slow for wave problems with discontinuous data [17].

2.3. Kernel and Gaussian-process methods

Gaussian processes are the standard low-dimensional regressor [18]. Batlle et al. develop a kernel framework with a priori error analysis and report it competitive with DeepONet and the Fourier neural operator [10]. A position paper argues that many neural-surrogate successes sit on low-dimensional manifolds where any competent reduced model interpolates well [19].

2.4. Fair comparison, tuning and reporting

McGreivy and Hakim found that 79% of 76 reviewed articles compared with weak baselines [7]. De Hoop et al. study the cost–accuracy trade-off of neural architectures [20]; Westermann et al. compare neural and polynomial surrogates by Pareto frontiers and find no universal winner [11]; a Common Task Framework has been proposed to evaluate scientific machine-learning methods against curated baselines [21]. Random search is an efficient alternative to grid search [22]; adaptive early-stopping schemes such as Hyperband [23] and software such as Optuna [24] reduce tuning cost; reporting results as a function of tuning budget and accounting for run-to-run variance have been advocated [25, 26].

2.5. Surrogate selection, learning curves and software

Williams and Cremaschi compared eight surrogate forms on generated data to look for rules for choosing a form [27]. Learning curves, the error as a function of training-set size, have been surveyed for decision making in supervised learning [28], and error scaling laws have been fitted across data scales in deep learning [29]. General-purpose libraries for model-order reduction [30] and uncertainty quantification [31] exist; we did not compare their features with ours, and the finite-element library `scikit-fem` [32] is used here only to generate independent test problems. PDEBench [33] provides large simulated benchmarks for scientific machine learning; we did not use it (section 7).

2.6. Comparison with prior work

Table 1 lists what each closest work contains, judged from the parts we read (mostly abstracts; Westermann et al. in large part).

Table 1: Closest related work. A dash means that we did not find the property in the parts we read (mostly abstracts); it does not prove absence.

Work	Kernel/GP vs neural	Tuning or baselines analysed	Ties reported	Selection guidance	Basis check, runs forecast	Open tool
Hesthaven & Ubbiali [2]	–	–	–	–	–	–
Guo & Hesthaven [4]	–	–	–	–	–	–
Bērziņš et al. [12]	✓	–	–	–	–	–
Batlle et al. [10]	✓	–	–	–	–	–
Westermann et al. [11]	✓	–	–	–	–	–
McGreivy & Hakim [7]	–	✓	–	–	–	–
Williams & Cremaschi [27]	–	–	–	✓	–	–
This paper	✓	✓	✓	✓	✓	✓

2.7. Motivations

(i) Method comparisons are reported on fixed problem sets, so a practitioner cannot tell whether a published ranking applies to their problem. (ii) The outcome of a comparison depends on tuning effort [7, 25], so a tool must make the tuning effort explicit and equal. (iii) Small differences are often ties, so a tool should report ties instead of a single winner. (iv) The accuracy of any POD-based model is bounded by the basis (proposition 2), which should be checked before tuning a regressor. (v) The practical question is the number of runs needed, which learning-curve extrapolation can answer only roughly and must be validated, not assumed.

3. Preliminary Concepts

Problem. A parametric simulation maps an input vector $\boldsymbol{\mu} \in [0,1]^p$ (rescaled to the unit box) to an output field $\mathbf{u}(\boldsymbol{\mu}) \in \mathbb{R}^m$ sampled on a fixed grid. We are given N runs $\{(\boldsymbol{\mu}_i, \mathbf{u}_i)\}_{i=1}^N$ and want a model $\hat{\mathbf{u}}$ that is accurate on new inputs. Accuracy is the mean relative L_2 error over n held-out inputs,

$$\operatorname{err}(\hat{\mathbf{u}}) = \frac{1}{n} \sum_{j=1}^{n} \frac{\|\hat{\mathbf{u}}(\boldsymbol{\mu}_j) - \mathbf{u}(\boldsymbol{\mu}_j)\|_2}{\|\mathbf{u}(\boldsymbol{\mu}_j)\|_2}. \tag{1}$$

POD basis. With $\bar{\mathbf{u}}$ the mean of the training fields and $V_r \in \mathbb{R}^{m \times r}$ the r leading left singular vectors of the centred snapshot matrix, a POD model has the form

$$\hat{\mathbf{u}}(\boldsymbol{\mu}) = \bar{\mathbf{u}} + V_r \, \hat{\mathbf{a}}(\boldsymbol{\mu}), \tag{2}$$

where $\hat{\mathbf{a}}$ regresses the coefficients $\mathbf{a} = V_r^\top (\mathbf{u} - \bar{\mathbf{u}})$ on $\boldsymbol{\mu}$.

Regressors. Gaussian process (GP): the posterior mean with a Matérn-5/2 or squared-exponential kernel and one length-scale per input (ARD), hyper-parameters fitted by maximising the marginal likelihood [18]. *Kernel ridge regression (KRR):* $\hat{a}(\boldsymbol{\mu}) = \sum_i \alpha_i k(\boldsymbol{\mu}, \boldsymbol{\mu}_i)$ with $\boldsymbol{\alpha} = (K + \lambda N I)^{-1}\mathbf{a}$. *Polynomial:* ridge regression on total-degree monomials.

Ridge hat matrix and leave-one-out. For a ridge-type smoother the fitted values are $H\mathbf{a}$ with $H = K(K + \lambda N I)^{-1}$, and the leave-one-out residual of run i is $(a_i - \hat{a}_i)/(1 - H_{ii})$ (proposition 1).

Tie band. Two errors are *tied* if their ratio lies within $10^{\pm 0.1} \approx 1.26$; the outcome of a comparison of groups A and B is $\bar{y} = \log_{10}(\mathrm{err}_B/\mathrm{err}_A)$, positive when A is better, and a cell is a tie if $|\bar{y}| \leq 0.1$.

Assumptions.

A1 Inputs of training and test runs are drawn independently from the same distribution on the unit box; extrapolation tests (outside the box) are reported separately.

A2 The output grid is the same for every run and the solver is deterministic (no noise).

A3 Runs are exchangeable, so that k-fold cross-validation estimates the error on new runs. This fails for adaptive or clustered designs; the tool does not detect that.

A4 The number of inputs is moderate (up to a few tens in our tests; only five of our 44 problems have ten or more).

4. Proposed Methodology: SurroSelect

4.1. Overview

SurroSelect takes the N runs and returns (a) the selected stand-in, (b) the set of models tied with it, (c) a basis check, and (d) a learning-curve forecast (fig. 1, algorithm 1). Every step uses only the training runs.

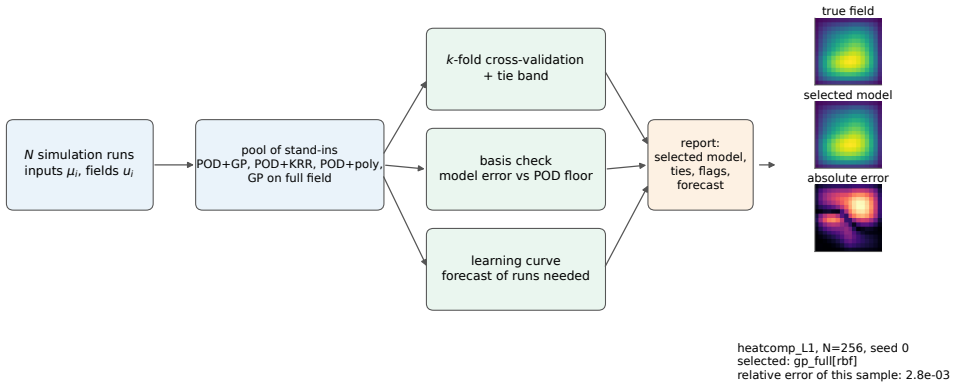


Figure 1: SurroSelect pipeline. The N simulation runs enter a pool of stand-in models; k-fold cross-validation selects the best model and reports which others are tied with it; the basis check compares the model error with the error of projecting held-out fields onto the POD basis; the learning curve forecasts the error for more runs. The example fields are taken from the first test sample of problem `heatcomp_L1` (seed 0, $N = 256$): true field, selected model, and absolute error, as chosen by the rule in section 6.5.

4.2. Candidate pool

The default pool has seven members, all classical: POD+GP with a Matérn-5/2 kernel and rank 32, with a squared-exponential kernel and rank 32, and with Matérn-5/2 and rank 8 (GP on the four leading modes with marginal-likelihood fitting, remaining modes sharing the fourth kernel); POD+KRR-wide, whose kernel, length-scale (0.05–$15\sqrt{p}$ on the unit box) and ridge (10^{-16}–10^{-2} times N) are chosen over a wide grid by the leave-one-out error of proposition 1; POD+polynomial, degree up to six with at most 3000 features, degree and ridge chosen by leave-one-out; and a GP directly on the full field (Matérn-5/2 or squared-exponential, shared ARD kernel fitted on at most 300 runs), included only when $m \leq 2000$. A tuned neural network (`TunedMLP`, section 5) can be added as an optional member.

4.3. Cross-validated, tie-aware selection

Each member is scored by k-fold cross-validation ($k = 4$ by default) with all fitting, including hyper-parameter fitting, repeated inside each training fold. The member with the smallest mean cross-validated error (1) is selected, and all members within the tie band of section 3 are reported as tied. Members that fail to fit receive infinite error.

Algorithm 1: SurroSelect (default settings)

Input: runs $(\boldsymbol{\mu}_i, \mathbf{u}_i)_{i \le N}$, folds k, tie band $\tau = 0.1$ decade, optional target error ε

Scale inputs to the unit box; split the runs into k folds;

foreach *pool member* c **do**

  fit c on each training fold and score it on the held-out fold; $e_c \leftarrow$ mean error (1);

$c^\star \leftarrow \arg\min_c e_c$; tied set $\leftarrow \{c : \ e_c \le 10^\tau e_{c^\star}\}$; refit $c^\star$ on all N runs;

Basis floor $\phi \leftarrow$ mean projection error of held-out fields onto a rank-32 POD basis; flag the basis if $e_{c^\dagger} \le 1.25\,\phi$ for the best POD-based $c^\dagger$;

Learning curve: errors of $c^\star$ at $n \in \{N/8, N/4, N/2, N\}$ (training-fold sizes); fit $\log e = \log A - b \log n$ on the last three points (and on all four); forecast $e(4N)$; if ε is given, return $(A/\varepsilon)^{1/b}$ unless $b < 0.1$ or the result exceeds $16N$;

return *$c^\star$, tied set, basis flag, forecast*

4.4. Basis check

On each fold, a rank-32 POD basis is fitted on the training part and the held-out fields are projected onto it; the mean relative error of this projection is the *basis floor*. If the best POD-based member's cross-validated error is within 25% of the floor, the tool flags the basis as limiting and recommends a full-field model, a larger basis, or a nonlinear output map. The reason is proposition 2.

4.5. Budget forecast

For the selected member, cross-validated errors are computed at 1/8, 1/4, 1/2 and all of the training folds. A power law $e(n) = A\,n^{-b}$ is fitted by least squares in log–log coordinates to the last three points (central forecast) and to all four (alternative); the lower and upper of the two forecasts form a range. The number of runs needed for a target error ε is $(A/\varepsilon)^{1/b}$; the tool declines to answer if $b < 0.1$ or if the answer exceeds sixteen times the current number of runs. This is an extrapolation. Section 6.3 evaluates the forecast of the error at four times the budget and the implied yes-or-no decision whether a target is reached within the 1024 runs of our design; the inversion that returns a number of runs was not evaluated against a finer budget grid.

4.6. Theory

Two results are standard and are included because the tool relies on them; proofs are in the Supplement (Sec. S1).

Proposition 1 (Closed-form leave-one-out)**.** *For a ridge smoother with hat matrix H (so $\hat{\mathbf{a}} = H\mathbf{a}$ on the training inputs), the leave-one-out prediction error at run i equals $(a_i - \hat{a}_i)/(1 - H_{ii})$. For $H = K(K + \lambda NI)^{-1}$ with $K = Q\,\mathrm{diag}(w)Q^\top$, $1 - H_{ii} = \sum_j Q_{ij}^2\, \lambda N/(w_j + \lambda N)$, which can be evaluated without cancellation when λN is tiny.*

Proposition 2 (Basis floor)**.** *Let $\hat{\mathbf{u}}$ be any model whose outputs lie in the affine subspace $\bar{\mathbf{u}}+\mathrm{span}(V_r)$. Then for every run, $\|\hat{\mathbf{u}}-\mathbf{u}\|_2 \geq \|(I-V_rV_r^\top)(\mathbf{u}-\bar{\mathbf{u}})\|_2$, so the mean relative error of $\hat{\mathbf{u}}$ is at least the mean relative projection error of the basis, whatever the regressor.*

The first is the PRESS identity for ridge regression [34]; the second is the optimality of orthogonal projection. The forecast has no guarantee: it is exact only if the error follows a power law, and section 6.3 tests how often it is useful.

5. Experimental Framework

5.1. Setup

Data protocol. For each problem we generate 4096 candidate training inputs, 1000 held-out test inputs and 500 extrapolation inputs (coordinates drawn from $[-0.25, 1.25]^p$, rejected if all lie inside the unit box) from streams seeded by the problem name; solutions are computed once and cached. A seed $s \in \{0, 1, 2\}$ fixes a permutation of the pool, and the training set for budget N is its first N elements, so training sets are nested within a seed. Budgets are $N \in \{32, \ldots, 1024\}$ for 24 problems and $\{64, 256, 1024\}$ for the others.

Problems. Table 2 lists the 44 problems. 36 come from our own solvers, each verified against exact or manufactured solutions or by grid refinement; they are grouped into six classes named after application areas (the classes are our own groupings of standard PDE families; nothing was calibrated to a real application). eight problems were added in the final stage and solved with the independent finite-element library `scikit-fem` (version 12.0.2); their unit tests compare with a series solution, a scaling law, load reversal, and mesh and time-step convergence. Eight further `scikit-fem` problems and one seconds-per-run problem were added afterwards for the held-out confirmation (section 6.7); they are not part of the 44 counted here. All data are simulated.

Table 2: The 44 test problems by class. "Inputs" is the number of parameters p, "Outputs" the number of field values m.

Class	Problems	Inputs	Outputs	Solver
Composite heat conduction	2	4–9	256	scikit-fem (third party)
Convection–diffusion	2	3–4	256	scikit-fem (third party)
Elastic L-shaped plate	2	4–7	130	scikit-fem (third party)
Transient heat, moving source	2	3–5	192	scikit-fem (third party)
Flow through porous rock	9	3–32	256	our code
Heat conduction	9	3–8	128–256	our code
Pollutant or heat carried by a flow	4	4	128–256	our code
Spreading fronts and reactions	4	3–4	128–256	our code
Thermal runaway	2	3	256	our code
Waves, shocks, acoustics	8	3–4	128–256	our code

Hardware and software. All runs used 4 CPU cores (Intel Xeon @ 2.10GHz), no GPU, Python 3.13.16, NumPy 2.5.3 [35], SciPy 1.18.1 [36], scikit-learn 1.9.1 [37], PyTorch 2.14.1 [38] and Optuna 5.0.0 [24]; each model fit used one thread.

5.2. Metrics

The error is (1). For outputs with a large offset (the real datasets, section 6.6) the normalised root-mean-square error is used instead,

$$\mathrm{NRMSE} = \frac{\|Y - \hat{Y}\|_F}{\|Y - \bar{Y}_{\mathrm{train}}\|_F}, \tag{3}$$

where Y and $\hat{Y}$ stack the held-out true and predicted outputs and $\bar{Y}_{\mathrm{train}}$ is the mean of the training outputs. The comparison between a group A (here the reduced-basis group) and a group B (the direct networks) is

$$\bar{y} = \frac{1}{S}\sum_{s=1}^{S} \log_{10} \frac{\mathrm{err}_B^{(s)}}{\mathrm{err}_A^{(s)}}, \tag{4}$$

over seeds, with the best member of each group chosen on validation data; $\bar{y} > 0.1$ means A is clearly better, $|\bar{y}| \le 0.1$ is a tie. For the tool, the *regret* of a cell (q, N) (problem q, budget N) and the *forecast ratio* of a case are

$$\mathrm{regret}(q,N) = \frac{\mathrm{err}(\mathrm{selected})}{\min_{c\in\mathrm{pool}} \mathrm{err}(c)}, \qquad \rho_{\mathrm{forecast}} = \frac{\hat{e}(4N_0)}{e_{\mathrm{actual}}(4N_0)}, \tag{5}$$

with all errors measured on the 1000 test inputs; the actual error is that of the same pool member refit on $4N_0$ runs, and the forecast is within a

factor two if $1/2 \le \rho_{\text{forecast}} \le 2$. The *reach-target classification* compares the statements $\hat{e}(1024) \le \varepsilon$ and $e_{\text{actual}}(1024) \le \varepsilon$ for $\varepsilon \in \{1\%, 0.1\%\}$ and reports the share of cases in which they agree, next to the share obtained by always giving the more common answer. Cell-wise values are geometric means over the three seeds.

5.3. Baselines

- *The five-method benchmark* ("the benchmark" below) under one tuning protocol: POD+GP, POD+polynomial, POD+MLP (reduced-basis group) and a direct MLP and a bottleneck MLP (network group). Each gets eight random-search trials (the first is a fixed default) scored on a random fifth of the training runs; networks use Adam for 1500 epochs in single precision.

- *Always POD+GP*: the benchmark's POD+GP, i.e. no selection; and *the best fixed pool member*, the single member with the smallest median regret (identified after the fact, hence an optimistic reference).

- *Oracle*: the best pool member by test error (an upper bound; unavailable in practice).

- *Strongly tuned network*: `TunedMLP`: Optuna TPE with successive-halving pruning [24, 23], 16 trials over output type (full field or POD rank 8/16/32), depth 2–4, width 64–256, activation, learning rate and weight decay; full-batch float64 training; final refit on all runs with four times the epochs and 200 L-BFGS iterations. Compared with the benchmark networks it has about twice the number of trials, four times the epochs, double precision and an L-BFGS polish; it is selected on a validation split of the training runs only, and it was run only at $N = 256$ and 1024 (16 cells).

Controls for the mechanism study are described in section 6.1.

5.4. Hypotheses and how each is tested

The hypotheses, metrics and pass/fail criteria below were committed to version control before the corresponding results (commit hashes in Supplement, Sec. S2); table 3 lists them. Earlier-stage hypotheses (pilot diagnostic, mechanism controls, longer training) are in the Supplement (Sec. S3). Binomial proportions are given with Wilson 95% intervals [39]; medians with percentile bootstrap intervals (2000 resamples over cells); the tests against the thresholds are one-sided exact binomial tests with Holm adjustment [40] over the three that admit one.

Table 3: Pre-specified hypotheses for the tool and their pass criteria (cells: 8 problems $\times$ $N \in \{64, 256, 1024\} = 24$).

ID	Statement	Pass criterion
H1a	Cross-validated selection is close to the oracle	median regret ≤ 1.5 and $\geq 75\%$ of cells ≤ 2
H1b	Not worse than always using POD+GP	median ratio ≤ 1
H1c	At least as good as the benchmark's own selection	median ratio ≤ 1; more cells better than worse by $> 10\%$
H2	The forecast is usable	$4N_0$ forecast within a factor 2 in $\geq 70\%$ of 48 cases; reach-target classification $\geq 75\%$
H3	The basis flag identifies cases where a full-field model is better	$\geq 75\%$ of flagged cells (if at least five)
H4	A tuned network rarely beats the tool	better by $> 25\%$ in at most 4 of 16 cells
H5	The benchmark finding holds on problems solved by third-party code	at least 5 of 8 problems reduced-basis-clear at $N = 1024$

6. Results and Analysis

All numbers in this section are written by the aggregation script from the result files. Cell-wise values are geometric means over three seeds; intervals are 95%.

6.1. Where each family wins, and by how much (the evidence base)

Main problems. Figure 2 shows the outcome $\bar{y}$ of (4) for the reduced-basis group against the direct networks. Among the 24 main problems (those with the full budget range), the reduced-basis group is clearly better in 5 problems at $N = 32$, the networks in 5, and 14 are ties; at $N = 1024$ the counts are 13, 8 and 3. The number of network wins at $N = 1024$ is 6 if the tie band is widened to 0.15 decade and 10 if it is narrowed to 0.05, so counts of ties and small wins are not stable. Where the reduced basis wins, it can win by orders of magnitude (panel b, top rows); where networks win, the largest margin is a factor of about three.

Accuracy reached. For a coarse target of 10% error, both groups reach it on all 36 own-solver problems within 1024 runs; for 1% they are about equal (27 against 28 problems); at the demanding end, 0.1%, the reduced-basis group reaches it on 19 problems and the networks on 10 (9 against 4 among the 24 main problems, excluding the eight smooth problems we added later because

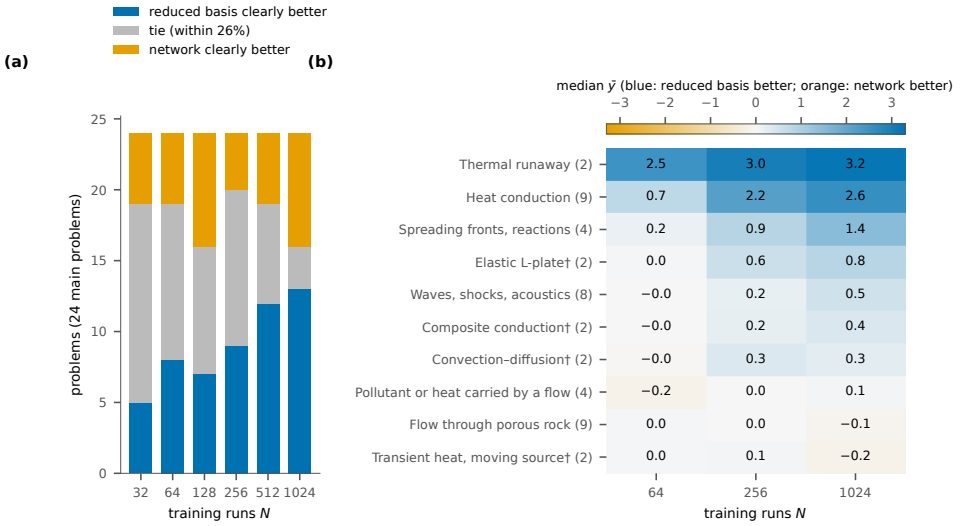


Figure 2: The reduced-basis group wins clearly on smooth problems at large budgets, and most differences at small budgets are ties. (a) Number of the 24 main problems (own solvers; five methods; three seeds; tie band ±0.1 decade) in which the reduced-basis group is clearly better, tied, or clearly worse than the direct networks, by training budget. (b) Median signed effect $\bar{y}$ by problem class over all 44 problems (number of problems in brackets; †: solved with `scikit-fem`); the colour scale is centred at zero and blue is the same colour as in (a). Per-problem values are in Supplement Table S3. The $N = 1024$ results are confounded with a doubling of the networks' optimiser steps (Supplement Sec. S3).

we expected the Gaussian process to do well). On our own problems a solve takes a median of 6.0 ms, so fitting a surrogate costs more than running the simulation 1024 times on most of them; the case for surrogates rests on simulations much slower than ours, and the comparison that matters is accuracy per run.

Problems solved by third-party code (H5). On the eight problems solved by `scikit-fem`, the reduced-basis group is clearly better at $N = 1024$ in three of eight; the criterion of at least five (H5) is therefore **not met**. The other problems are ties or network wins (Table S3): the networks are ahead on the composite-conduction problem with nine inputs and high contrast and on both moving-source problems. The simple picture "reduced basis wins at large budgets" therefore does not carry over to five of these eight problems; the tool of section 4 is designed for exactly this case, and section 6.3 tests it.

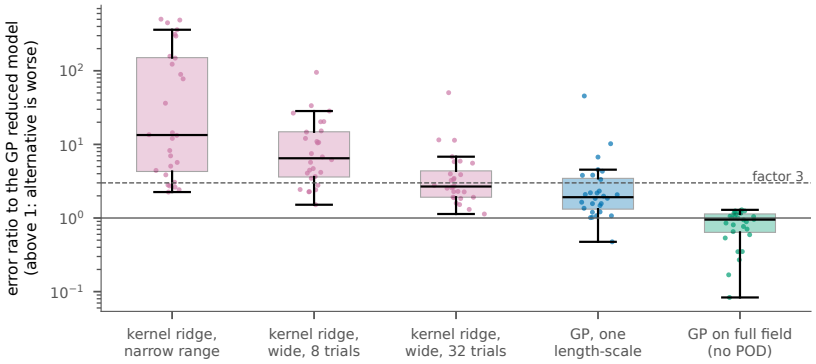


Figure 3: Widening the hyper-parameter search of kernel ridge closes most of the gap to the Gaussian-process reduced model, and a Gaussian process on the full field is as good as the POD-based one. Error ratio of each alternative to the reduced-basis Gaussian process in the 28 budget–problem combinations where that model clearly beats the best network (own-solver problems, budgets 64, 256, 1024); boxes show quartiles, dots individual combinations, the dashed line a factor of three.

6.2. Why the Gaussian-process reduced model wins when it does

Figure 3 compares alternatives with the Gaussian-process reduced model in the 28 budget–problem combinations (budgets 64, 256, 1024) where it clearly beats the best network (the comparison uses the best network by *test* error, an oracle that favours the networks). Kernel ridge with the narrow range of ridge and length-scale of our first control is a median 13 times less accurate; with a wide range and the same eight tuning trials, 6.5 times; with 32 trials, 2.7 times (57% of cells within a factor three), and the wide kernel ridge beats the best network clearly in 64% of these cells. A Gaussian process on the full field without POD is as accurate as the POD-based one (median ratio 0.95). The Gaussian-process advantage is therefore largely an advantage of a smooth regressor whose regularisation is searched widely and finely, and not of the reduced basis; we did not isolate the contribution of marginal-likelihood fitting, because its cost is not matched by the kernel-ridge search. The wide-range control was added after an AI-agent review of an earlier draft showed that our first control could not reach the regularisation that near-interpolating fits need; this is a post hoc control (Supplement, Sec. S2). Three further results qualify the picture. Training the networks four times longer moved the verdict in 15 of 48 cases, always toward the networks, and improved 90% of the refits; the large gaps survived stronger training (double precision, 6000 steps, L-BFGS) in 5 of 7 decisive problems, with the networks' error floor lowered by a median 0.47 decade to about 2.0×10^{-4}.

Table 4: Pre-specified criteria (table 3) and results on the eight problems solved by third-party code. Intervals are 95% (bootstrap for medians, Wilson for proportions, both treating cells as independent); problem-level intervals are in the text. The Holm-adjusted p-values are one-sided exact binomial tests against the thresholds.

ID	Criterion	Observed (95% interval)	Holm p	Verdict
H1a	median regret ≤ 1.5; $\geq 75\%$ of cells ≤ 2	median 1.00 [1.00, 1.02]; 100% [86%, 100%]	0.003	met
H1b	median ratio vs always-GP ≤ 1	0.85 [0.59, 0.97]; better/worse 15/1	–	met
H1c	median ratio vs benchmark selection ≤ 1; better $>$ worse	1.01 [0.59, 1.19]; better/worse 11/9	–	not met
H2	forecast within $\times 2$ in $\geq 70\%$ of 48	37 of 48 (77% [63%, 87%])	0.363	met
H2 (reach 1%)	classification accuracy $\geq 75\%$	100%	–	met
H2 (reach 0.1%)	classification accuracy $\geq 75\%$	88%	–	met
H3	$\geq 75\%$ of flagged cells (if ≥ 5)	15 flagged; 15 better (100%)	–	met
H4	strong network better by $> 25\%$ in ≤ 4 of 16	4 of 16	0.630	met
H5	reduced basis clear on ≥ 5 of 8	3 of 8	–	not met

On the extrapolation test set the median $\bar{y}$ at $N = 1024$ is -0.03 instead of 0.16. And the class of a cell agrees between halves of the random-search draws in 83% of cases (sign of clear differences: 99%), so small differences should not be over-interpreted. The earlier-stage hypotheses, including the pilot diagnostic that was not confirmed, are in the Supplement (Sec. S3).

6.3. Evaluation of SurroSelect on problems solved by third-party code

The tool was run on the eight `scikit-fem` problems at $N \in \{64, 256, 1024\}$ with three seeds (24 cells), and every pool member was also refit and scored on the 1000 test inputs, so that the *oracle* (the best pool member by test error; unavailable in practice) is known. Table 4 lists the pre-specified criteria and what was observed; table 5 gives the errors per problem. Only eight problems from four families are involved, and the three seeds of a problem share the pool and the test set, so cells are not independent; table 4 therefore also gives problem-level intervals (resampling the eight problems), and the verdicts describe these eight problems only. The tuned network was run only at $N = 256$ and 1024 (16 cells); fig. 5 shows it next to the tool.

Selection quality (H1a, H1b, H1c). Cross-validated selection chose the oracle member in most cells, and the regret was at most 1.18 in all 24 cells: median regret 1.00 (interval 1.00–1.02), 100% of cells within a factor two (Wilson interval 86%–100%; H1a met). Two caveats weaken this result. First, a fixed rule also passes the criterion: always choosing the single pool member

`gp_full[rbf]` gives a median regret of 1.14 and puts 75% of the cells within a factor two. The tool improves on that fixed choice by more than 10% in 12 cells and loses by more than 10% in two (median ratio 0.91). Choosing a member at random would give a median regret of 3.77 and 38% of cells within a factor two, so the tool is far from chance, but the fair reference is the best fixed member. That member was identified with hindsight; a fixed member chosen without it (the one with the smallest median regret on the *other* seven problems) gives a median regret of 1.69 and only 54% of cells within a factor two, so selection from the data is worth more than the comparison with the hindsight member suggests. Second, at the problem level, all eight problems have every cell within a factor two, yet a Wilson interval over eight problems has a lower limit of 68%, below the 75% threshold; the problem-level interval of the median regret is 1.00–1.01. The tool was no worse than always using the benchmark's POD+GP (median ratio 0.85, interval 0.59–0.97, problem level 0.59–0.98; better by more than 10% in 15 cells, worse in one; H1b met), which is nearly certain by construction because that model is a pool member. Against the benchmark's own selection among the five methods, which includes neural networks, the median ratio was 1.01 (interval 0.59–1.19, problem level 0.59–1.30); the tool was better by more than 10% in 11 cells and worse in nine, so H1c is **not met**. The cells where the tool loses (largest ratio 2.2 on `heatcomp_L2` at $N = 1024$; 1.4 on `lshape_L2` at $N = 1024$) are those in which a network from the benchmark beats every member of the default pool; the default pool contains no network.

Budget forecast (H2). For the 48 cases, the forecast from N_0 runs of the error at $4N_0$ runs was within a factor two of the actual error in 37 (77%, Wilson interval 63%–87%, problem level 65%–90%; met), but the interval includes the threshold of 70% (one-sided exact binomial $p = 0.18$ against it, Holm-adjusted 0.363). The pre-specified pooled criterion was met only because of the small budget: 92% of the cases from $N_0 = 64$ but only 15 of 24 (62%) from $N_0 = 256$, which is below the threshold, and 7 of the forecasts from 256 runs were too low by more than a factor two (fig. 4). Overall 7 forecasts were too low and 4 too high, and all misclassifications at the 0.1% target were optimistic. Errors that fall faster than a power law over the last data points are extrapolated too optimistically. The range between the two fits contains the actual error in only 12% of the cases and must not be read as an uncertainty interval. For the decision "will 1% (0.1%) be reached at 1024 runs?" from 256 runs, the forecast was right in 100% (88%) of 24 cases; always guessing the more common answer would be right in 75% (50%), so the 1% result carries no information beyond the base rate and the 0.1%

Table 5: Test error (geometric mean over three seeds) on the eight problems solved by third-party code: SurroSelect, the oracle (best pool member by test error), always POD+GP, the benchmark's validation-selected method, and the tuned network (only $N = 256$ and 1024).

Problem	N	Tool	Oracle	Always GP	Benchmark sel.	Tuned net
convdiff_L1	256	4.2×10^{-4}	4.2×10^{-4}	1.6×10^{-3}	1.6×10^{-3}	2.4×10^{-3}
convdiff_L2	256	1.1×10^{-2}	1.1×10^{-2}	1.5×10^{-2}	1.5×10^{-2}	1.1×10^{-2}
heatcomp_L1	256	2.2×10^{-3}	2.0×10^{-3}	2.0×10^{-3}	2.0×10^{-3}	3.7×10^{-3}
heatcomp_L2	256	1.6×10^{-1}	1.6×10^{-1}	1.9×10^{-1}	1.0×10^{-1}	1.2×10^{-1}
heatmove_L1	256	9.0×10^{-4}	9.0×10^{-4}	7.5×10^{-3}	7.5×10^{-3}	5.5×10^{-3}
heatmove_L2	256	2.1×10^{-2}	2.1×10^{-2}	3.2×10^{-2}	3.2×10^{-2}	2.7×10^{-2}
lshape_L1	256	8.5×10^{-4}	8.5×10^{-4}	8.5×10^{-4}	8.5×10^{-4}	6.2×10^{-3}
lshape_L2	256	7.6×10^{-2}	7.6×10^{-2}	7.4×10^{-2}	5.3×10^{-2}	3.7×10^{-2}
convdiff_L1	1024	1.4×10^{-5}	1.4×10^{-5}	1.0×10^{-3}	1.0×10^{-3}	2.1×10^{-3}
convdiff_L2	1024	1.9×10^{-3}	1.8×10^{-3}	8.3×10^{-3}	8.3×10^{-3}	5.6×10^{-3}
heatcomp_L1	1024	1.2×10^{-4}	1.2×10^{-4}	4.3×10^{-4}	4.3×10^{-4}	2.1×10^{-3}
heatcomp_L2	1024	8.1×10^{-2}	8.0×10^{-2}	9.0×10^{-2}	3.7×10^{-2}	3.6×10^{-2}
heatmove_L1	1024	9.1×10^{-5}	9.1×10^{-5}	6.6×10^{-3}	4.5×10^{-3}	3.5×10^{-3}
heatmove_L2	1024	4.2×10^{-3}	4.2×10^{-3}	2.1×10^{-2}	1.5×10^{-2}	1.3×10^{-2}
lshape_L1	1024	7.1×10^{-5}	6.1×10^{-5}	1.2×10^{-4}	1.2×10^{-4}	3.6×10^{-3}
lshape_L2	1024	2.8×10^{-2}	2.8×10^{-2}	2.5×10^{-2}	2.0×10^{-2}	1.7×10^{-2}

result is above the base rate but not significant against the 75% threshold (one-sided $p = 0.12$).

Basis check (H3). The flag was raised in 15 of the 72 fits, which are five distinct (problem, budget) cells from four problems (convection–diffusion and transient heat); in all of them a full-field Gaussian process had a test error more than 10% below the best POD-based member (H3 met; the comparison is by test error over five POD members and two full-field members). This evidence is thinner than the count suggests. The flag was raised in only 47% of the 32 fits in which a full-field member was better by more than 10%, so its recall is low, and in all 15 flagged fits the cross-validated selection had already chosen a full-field member, so on these data the flag explained a decision rather than changed it. The flagged problems are those where the benchmark found networks competitive with the POD-based Gaussian process, which is consistent with proposition 2.

Against a tuned network (H4). The tuned network (Optuna, 16 trials; compared with the benchmark networks it also has four times the epochs, double precision and an L-BFGS polish, and about twice the trials) was better than the tool-selected model by more than 25% in four of 16 cells (H4 met, exactly at the limit of four). These four cells come from two problems (composite conduction and the L-shaped plate, both with seven or nine inputs, at two

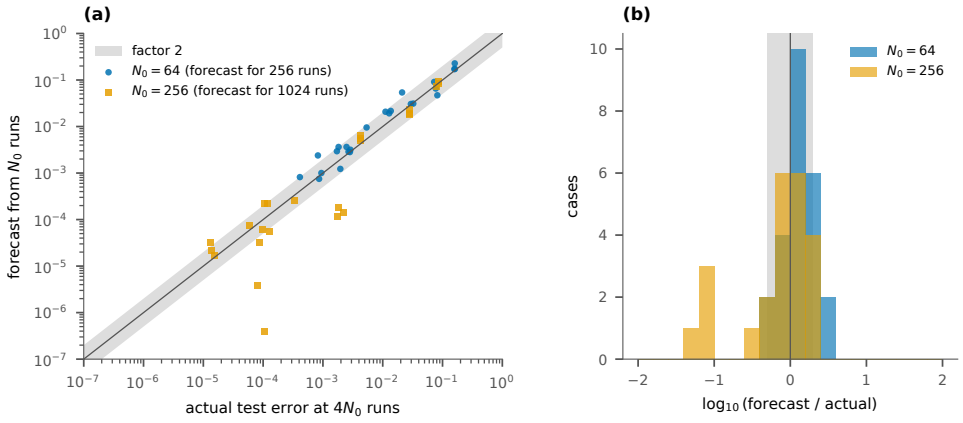


Figure 4: The learning-curve forecast is useful from small budgets and too optimistic from larger ones. (a) Forecast against actual test error at four times the starting budget (48 cases; problems solved by third-party code; shaded: within a factor two). (b) Histogram of $\log_{10}$(forecast/actual); cases below -0.3 are forecasts that are too low.

budgets each), and the Wilson interval for their share, 10%–49%, is wide. The median ratio of the tool's error to the tuned network's was 0.34; the tool was better by more than 10% in 11 cells and the network in 4. The tuned network improved on the benchmark's networks in 15 of 16 cells (median error ratio 0.64), but on the smooth problems at $N = 1024$ its error was still up to 152 times that of the tool, so it is probably under-trained and H4 says little about what a well-tuned network could reach. It needed a median of 146 s ($N = 256$) and 429 s ($N = 1024$) on one core, against 112 and 312 s for the tool's cross-validated selection (section 6.4). The tool's advantage over the tuned network is largest on the smooth problems, and the network wins on the two problems with seven or nine inputs (fig. 5).

6.4. Ablation, noise and cost

Ablation. Restricting the pool, with selection by the stored cross-validation errors (table 6), shows which members matter: without the full-field Gaussian process the regret is within a factor two in only 75% of cells, and without any Gaussian-process member (ridge and polynomial only) the test error is a median 2.7 times larger than with the full pool.

Sampling noise. A pool member picked at random gives median regret 3.77, and the seed-to-seed spread of the tool's regret is small (median regret 1.01, 1.00 and 1.00 at the three budgets), so the observed regrets are not explained

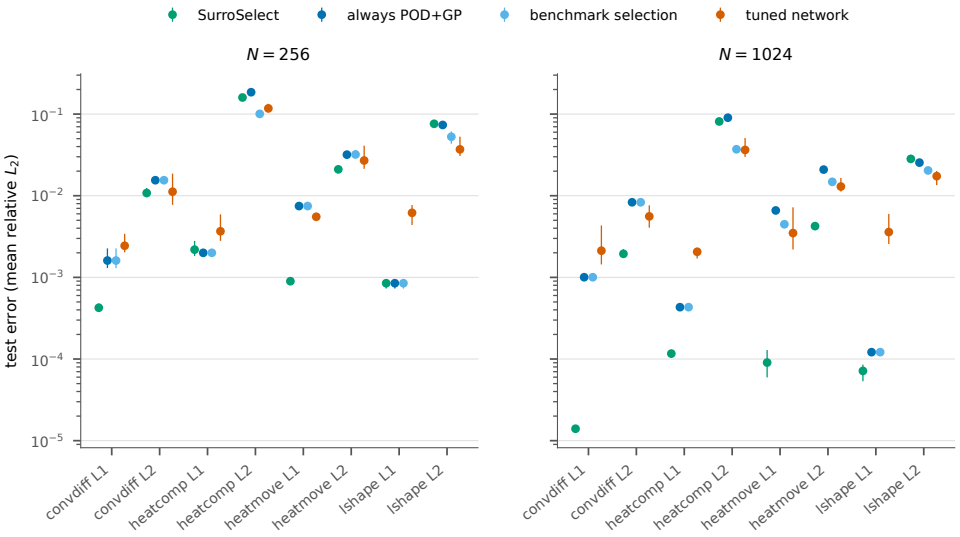


Figure 5: Test error of the tool and its competitors on each problem solved by third-party code. Points: geometric mean over three seeds; bars: minimum to maximum over seeds. The tuned network was run at $N = 256$ and 1024 only.

Table 6: Ablation of the pool on the eight problems solved by third-party code (24 cells); regret as in section 5.

Pool	Median regret	Cells with regret ≤ 2	Median error vs full pool
full pool	1.00	100%	1.00×
POD-based members only	1.07	75%	1.02×
no full-field GP	1.07	75%	1.02×
GP members only	1.01	96%	1.00×
no GP (ridge, polynomial)	2.81	33%	2.74×

by chance. Seeds share the pool and test set and nest their training subsets, so seed variability understates the variability of a new sample of runs.

Computational cost. Measured on 4 cores (Intel Xeon @ 2.10GHz), one thread per fit, selection with cross-validation took a median of 7, 112 and 312 s at $N = 64$, 256 and 1024; adding the learning curve brought this to 9, 134 and 426 s. The solvers of these problems take 3–27 ms per run, so for them the tool costs more than running the simulation 1024 times; its use is justified when one run takes minutes or hours.

Complexity. Let N be the number of runs, m the number of outputs, p the number of inputs, r the number of POD modes and k the number of

folds. The dominant costs are: an exact Gaussian process with fixed hyper-parameters, $O(N^3+N^2r)$ per fit; hyper-parameter fitting, which uses at most 300 runs and costs $O(n_{\text{opt}}^3 p)$ per optimiser step on the leading modes; kernel ridge, one eigendecomposition per kernel and length-scale, $O(N^3)$ each, with the whole ridge grid scored in $O(N^2)$ each; the polynomial, one singular value decomposition of the $N \times F$ design matrix with $F \leq 3000$, $O(NF^2)$; the full-field Gaussian process, as the POD-based one with all m outputs sharing one kernel, $O(N^3 + N^2 m)$; the POD, $O(Nm \min(N, m))$. Cross-validation multiplies each by k, and the learning curve adds about $\sum_j k\, c(N/2^j)$, i.e. roughly one further cross-validated fit of the selected member. Between $N = 256$ and 1024 the measured median selection time grew by a factor of about 2.8, against the factor 64 predicted by cubic scaling, because the hyper-parameter fit is capped at 300 runs.

6.5. Qualitative analysis

Figure 6 shows the first test sample in dataset order for the first problem of each family at $N = 256$ (seed 0), and failure cases selected by rules fixed before the figure was made: the problem with the largest selection regret at $N = 256$ and the problem with the largest test error (a problem that is already shown is not repeated). The selected model reproduces the large-scale structure of the fields in the rule-selected examples, with relative errors given beside each panel; the error maps show structure rather than noise, so the remaining error is systematic. In the failure case with the largest error (the nine-input, high-contrast composite-conduction problem) the error is spread over the whole field, so no member of the pool captures this map at 256 runs; this is consistent with the benchmark, where networks beat the reduced-basis group on this problem.

6.6. Comparative analysis and real-data check

Comparison with the benchmark. Together with section 6.1: on smooth problems with few inputs a Gaussian-process or well-regularised kernel member of the pool is hard to beat; where the POD basis limits accuracy a full-field Gaussian process is the better choice; and where networks from the benchmark win (the composite-conduction problem with nine inputs), the default pool cannot match them (H1c). The tool's contribution is to find these cases from the training runs alone. In a descriptive, design-informed run on the 36 earlier problems (one seed, $N = 256$; Supplement, Sec. S7), the median regret was 1.00 but the largest was 9.25 (`fisher_L1` (9.2), `advdiff_L1` (1.7), `incl3_L1` (1.2)); that is, the tool can fail badly on a problem, and the failure

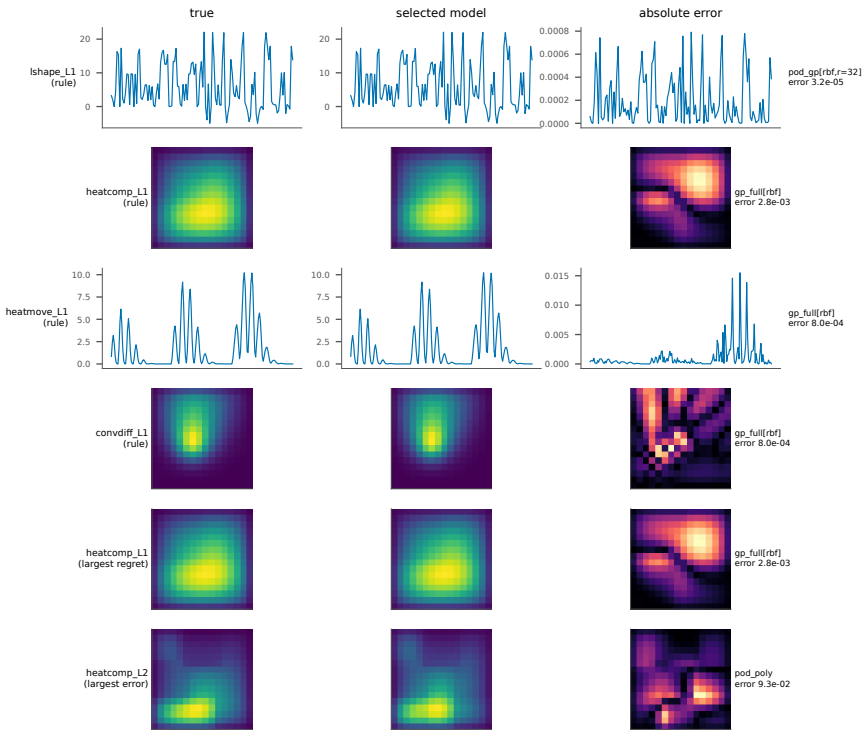


Figure 6: The selected model reproduces the large-scale structure of single test fields; its errors are systematic. Predictions of the selected model on single test samples chosen by a fixed rule (first test sample in dataset order, first problem of each family, plus the failure cases defined in section 6.5). Columns: true field, selected model, absolute error; the L-shaped plate has no regular grid and is shown as a profile over probe nodes.

is not visible in the cross-validated report. Those problems were used to design the pool, so the run is not evidence for the criteria.

Real measured data. Two real datasets were used (`data_real/`, CC BY 4.0): wind-tunnel measurements of airfoil noise (1,503 rows, five inputs) and the output of a combined-cycle power plant (9,568 rows, four inputs). Both have a scalar, noisy output, so they violate assumption A2, are not field data, and make the POD rank one, so the data say nothing about the POD-versus-full-field choice that the tool is built around; the normalised RMSE replaces the relative error. With the protocol of Supplement addendum v3.1 (committed before any model was fitted to these data), median regret was 1.04 over six cells (R1 met), the tool was no worse than always-POD+GP in five of six cells (R2 met; two of these successes are exact ties, because the same model was chosen), the tuned network was better by more than 25% in zero cells

Table 7: Real-data check: normalised RMSE of the tool, the oracle, always-POD+GP and the tuned network (geometric mean over three seeds).

Dataset	N	Tool	Oracle	Always GP	Tuned net	Regret
airfoil	64	0.5960	0.5218	0.5868	0.6792	1.14
airfoil	256	0.3585	0.3578	0.3585	0.4072	1.00
airfoil	1024	0.2107	0.1845	0.2107	0.2037	1.14
ccpp	64	0.2867	0.2676	0.3153	0.5854	1.07
ccpp	256	0.2554	0.2539	0.2920	0.3682	1.01
ccpp	1024	0.2453	0.2437	0.2500	0.3530	1.01

(R3 met), and 9 of 12 forecasts were within a factor two (R4 met). We had predicted that R4 would fail because of measurement noise; it did not. The airfoil pool holds only 1,052 rows, so its $N = 1024$ results use nearly all of it. A review found a shape bug in the tool for single-output problems (deviation V13); after fixing it, the tool part of all 18 real-data runs was repeated, and 18 of 18 selections and all selected errors were unchanged. Table 7 lists the errors.

Plain-file validation and case study. The three command-line commands were run on exported files for three problems; all sequences ran, and the error printed by `evaluate` matched an independent computation (Supplement, Sec. S8). Figure 7 shows the report for problem `heatmove_L1` with 256 runs and a target error of 0.1%.

6.7. Held-out confirmation, a network in the pool, and an expensive solver

The criticisms that remained after the evaluation above were that only eight problems from four families were involved, that the tuned network may have been under-trained, that the default pool has no network, and that the solvers are so fast that the tool's cost was never weighed against a real saving. We therefore froze the tool, wrote a second protocol with criteria *and predictions* (addendum v3.2; the freeze commit is recorded in Supplement, Sec. S2), and only then fitted anything to eight further `scikit-fem` problems (table 8; four new families: Helmholtz near resonance, nonlinear reaction–diffusion solved by Newton's method, elasticity on a disc, advection in a cellular flow). Unit tests of the new solvers exposed a boundary-region bug in one of them before any model was fitted (deviation V14). The pool, grids and criteria are those of the first evaluation; the problems were chosen for physical variety and two contain deliberately hard regimes.

```
# SurroSelect report
256 runs, 3 inputs, 192 outputs. Cross-validated mean relative L2 error (4
    folds).

| model | CV error | within tie band of best | seconds |
|---|---:|:---:|---:|
| gp_full[rbf] | 1.424e-03 | yes | 24.1 |
| gp_full[matern52] | 2.432e-03 |  | 19.6 |
| pod_gp[matern52,r=32] | 7.841e-03 |  | 7.2 |
| pod_gp[rbf,r=32] | 8.153e-03 |  | 7.3 |
| pod_krr_wide | 1.222e-02 |  | 0.7 |
| pod_poly | 2.017e-02 |  | 0.1 |
| pod_gp[matern52,r=8] | 6.331e-02 |  | 6.7 |

**Selected:** gp_full[rbf] (CV error 1.42e-03). Models tied with it (within
    1.26x): gp_full[rbf].

**Basis check.** Best POD-based model: pod_gp[matern52,r=32] (CV error
    7.84e-03); error of projecting held-out fields on a 32-mode basis: 7.53e-03.
    The best POD-based model is within 25% of the best error any 32-mode POD
    basis can reach: the basis, not the regressor, limits accuracy. Use a full-
    field model, a larger basis, or a nonlinear (autoencoder) output map.

**Learning curve** (gp_full[rbf]): n=24: 1.71e-01, n=48: 2.15e-02, n=96:
    6.19e-03, n=192: 1.42e-03
Extrapolated error with 1024 runs: about 5.6e-05 (range 3.0e-05 to 5.6e-05);
    extrapolation over 4x the data is a rough guide only.
Runs needed for error 0.001: about 235.
```

Figure 7: SurroSelect report (in Markdown) for `heatmove_L1` (256 runs, seed 0, target error 0.1%), unedited except for line wrapping.

Held-out result. Table 9 repeats the criteria of table 3 on these eight problems; six of the eight verdicts were met. Selection quality held up (H1a: median regret 1.04, 96% of cells within a factor two; the worst case, `helmholtz_L1` at $N = 1024$, is a regret of 5.2), and the tool was not worse than always using one Gaussian process (H1b, 0.97). Three results went the other way, and two of them against what we had predicted. (i) The forecast of H2 failed: 28 of 48 forecasts were within a factor two (58%; interval 44% to 71%), 18 were too optimistic by more than a factor two (median predicted/actual 0.58), and from 64 runs the share was only 54%, where we had predicted that it would hold. The classification of whether a target is reached was nevertheless right (100% for 1%, 88% for 0.1%, against 62% and 54% for always predicting the majority class). (ii) The tool again did not match the benchmark's own selection (H1c: median ratio 1.08, better in 6 cells and worse in 10; 5 of the worse cells are Helmholtz problems, where the benchmark's neural-network methods (POD with a network, direct network, bottleneck) were selected and beat the tool). (iii) The basis flag never fired (H3 not evaluable), although we had predicted that it would on the resonant problem. On the other side,

Table 8: The eight further problems (`scikit-fem`); p inputs, m outputs.

Problem	p	m	Description
`helmholtz_L1`	3	256	Helmholtz, k^2 in [2,20], source position
`helmholtz_L2`	4	256	Helmholtz, k^2 in [30,70] (several resonances), source position and width
`reacdiff_fe_L1`	4	256	nonlinear reaction-diffusion (Newton), uniform D
`reacdiff_fe_L2`	6	256	nonlinear reaction-diffusion (Newton), 2x2 blocks of D, strong reaction
`disc_elast_L1`	5	82	plane stress on a disc, 4 angular stiffness sectors + load angle
`disc_elast_L2`	9	82	plane stress on a disc, 8 angular stiffness sectors, high contrast
`advdiff_cell_L1`	3	256	advection-diffusion in a cellular flow
`advdiff_cell_L2`	4	256	advection-diffusion in a cellular flow, thin layers, source width

Table 9: Criteria on the first eight problems, on the eight further problems (tool frozen before they were run) and, as numbers only, pooled over sixteen. Each cell gives the observed value and the verdict; "better/worse" counts cells that differ by more than 10%.

ID	Criterion	First eight problems	Further eight (held out)	Pooled, 16 (numbers only)
H1a	median regret ≤ 1.5; $\geq 75\%$ of cells ≤ 2	1.00; 100%; met	1.04; 96%; met	1.01; 98%
H1b	median ratio vs always-GP ≤ 1	0.85; met	0.97; met	0.93
H1c	median ratio vs benchmark selection ≤ 1; better/worse cells	1.01; 11/9; not met	1.08; 6/10; not met	1.02; 17/19
H2	forecast within $\times 2$ in $\geq 70\%$	77%; met	58%; not met	68%
H2 (reach)	classification accuracy $\geq 75\%$ (1% / 0.1%)	100% / 88%; met	100% / 88%; met	100% / 88%
H3	basis flag helps (if ≥ 5 flagged)	15 flagged; met	0 flagged; not evaluable	15 flagged
H4	tuned network better by $> 25\%$ in ≤ 4 of 16	4 of 16; met	2 of 16; met	6 of 32
H5	reduced basis clear on ≥ 5 of 8	3 of 8; not met	6 of 8; met	9 of 16

the reduced-basis group was clearly better at the largest budget on six of the eight problems (H5 met), where we had predicted that it would not be, and the tuned network beat the tool by more than 25% in only two of 16 cells (H4 met, median error ratio tool/network 0.32). Problem-level 95% intervals (resampling the problems) for the medians are 1.00 to 1.11 (H1a), 0.91 to 1.04 (H1b) and 0.94 to 1.32 (H1c). Over all sixteen problems the forecast was within a factor two in 68% of 96 cases (problem-level interval 56% to 79%), below the not met threshold; we therefore describe the forecast as a rough guide that tends to be optimistic, not as validated.

A network in the pool. Adding the tuned network to the pool (cross-validated like every other member, $N = 256$, table 10) changed the choice in 5 of 24 fits and lowered the error clearly in two problems (`helmholtz_L1`, `reacdiff_fe_L2`; for `helmholtz_L1` from 0.64 to 0.36, where the network

Table 10: Test error (mean relative L_2, geometric mean over three seeds) at $N = 256$ on the eight further problems. "Tool + net" is the tool with a tuned network added to its pool.

Problem	Tool	Oracle	Always GP	Benchmark sel.	Tuned net	Tool + net
`advdiff_cell_L1`	1.0×10^{-4}	1.0×10^{-4}	2.9×10^{-4}	2.9×10^{-4}	2.9×10^{-3}	1.0×10^{-4}
`advdiff_cell_L2`	1.6×10^{-3}	1.4×10^{-3}	1.8×10^{-3}	1.8×10^{-3}	5.9×10^{-3}	1.6×10^{-3}
`disc_elast_L1`	2.8×10^{-3}	2.6×10^{-3}	2.1×10^{-3}	2.1×10^{-3}	8.7×10^{-3}	2.8×10^{-3}
`disc_elast_L2`	7.5×10^{-2}	7.4×10^{-2}	6.8×10^{-2}	6.8×10^{-2}	8.8×10^{-2}	7.5×10^{-2}
`helmholtz_L1`	6.4×10^{-1}	5.8×10^{-1}	1.4×10^{0}	2.3×10^{-1}	2.7×10^{-1}	3.6×10^{-1}
`helmholtz_L2`	1.0×10^{0}	9.7×10^{-1}	1.4×10^{0}	9.2×10^{-1}	2.0×10^{0}	1.0×10^{0}
`reacdiff_fe_L1`	1.4×10^{-4}	1.4×10^{-4}	1.6×10^{-4}	1.6×10^{-4}	1.2×10^{-3}	1.4×10^{-4}
`reacdiff_fe_L2`	4.3×10^{-3}	4.1×10^{-3}	4.4×10^{-3}	4.5×10^{-3}	3.9×10^{-3}	3.6×10^{-3}

alone gave 0.27). The enlarged pool met both of its criteria: median regret 1.05 against the best member of the enlarged pool (largest 1.13, 100% of cells within a factor two), and in seven of eight cells within 10% of the better of the tool alone and the network alone. The price is time: a median of 431 seconds per fit, against about a hundred seconds for the default pool at this budget. The network is therefore offered as an option (`-with-network`), not as part of the default pool.

A larger tuning budget for the network. With three times the trials, depths up to five and widths up to 512, the network improved on the standard one in 12 of 16 problems at $N = 256$ (median error ratio 0.85; 10 improved by more than 10%), at a median of 9 minutes per fit. It beat the tool by more than 25% in four of the 16 problems (`convdiff_L2`, `heatcomp_L2`, `lshape_L2`, `reacdiff_fe_L2`), against three for the standard network (`heatcomp_L2`, `helmholtz_L1`, `lshape_L2`). In the median problem the tool's error was about half that of the stronger network (median ratio tool/network 0.56). A larger search thus made the network somewhat better and did not change the picture: the network wins mostly on the level-two problems with more inputs or stronger nonlinearity, the tool elsewhere. We did not try larger budgets than these, so a network tuned further could do better.

An expensive solver. The L-shaped plate of the first evaluation was solved on a finely refined mesh, which takes 1.52 seconds per run on this machine (table 11); 512 training runs and 300 test runs were computed once. This is still far from the hours-long simulations that motivate surrogates, but it is slow enough for a measured trade-off. Selection stayed close to the oracle (mean regret 1.28 at 64 runs and 1.35 at 256), and the error fell from 3.7×10^{-2} to 8.2×10^{-4} between 64 and 256 runs, better than always using one Gaussian process (4.4×10^{-3}) and than the tuned network (6.2×10^{-3},

Table 11: Cost of the surrogate for the finely meshed plate (one solver run: 1.52 seconds). The tool time includes the cross-validated fit and the learning curve.

	$N = 64$	$N = 256$
Simulation time for the training runs (hours)	0.03	0.11
Tool time (seconds)	9	120
Predictions after which the surrogate is cheaper than the solver	70	335

96 seconds). The forecast from 64 to 256 runs was within a factor two in only one of three seeds (ratios 1.25 to 3.14, all too high), so the criterion for the expensive case was not met. Counting the simulations, the tool took 120 seconds at 256 runs, next to 0.11 hours of solver time to compute those runs; the surrogate is cheaper than the solver after about 335 predictions, and ten thousand predictions would cost 4.2 hours of solver time against 0.14 hours with the surrogate (including the training runs and the tool). This is a measurement for one problem; it is not evidence about simulations that take hours.

7. Discussion

What the evidence supports. On the problems solved by third-party code, cross-validated selection from a small pool of classical models found the oracle or a near-oracle member (H1a; a fixed choice of one full-field Gaussian process does almost as well), was not worse than always using a Gaussian process (H1b; nearly guaranteed because that model is in the pool), raised the basis flag only in cases where a full-field model was better but with low recall (H3), and gave usable forecasts of the error at four times the budget when starting from 64 runs on the first eight problems (H2). The tool was then frozen and run on eight further problems under criteria and predictions committed beforehand (section 6.7): selection quality, the comparison with always using one Gaussian process, the classification of whether a target is reached and the comparison with the tuned network held again, which supports these four claims beyond the eight problems on which the tool was developed. Against a tuned network the tool met the criterion of H4 only at its limit, and that network is probably under-trained. On real measured data with scalar, noisy outputs, all four descriptive criteria were met, although two of the five R2 successes are ties and the scalar outputs make the POD rank one, so these data do not test the central design of the tool.

What it does not support. The benchmark finding that the reduced-basis group wins at large budgets did *not* hold on five of the eight problems (H5 not met), so a fixed recommendation "use a reduced-basis Gaussian process" is not safe; the case for the tool is that it finds out from the data. The tool did not match the benchmark's own selection among five methods that include networks (H1c not met): where networks win, the default pool, which has none, loses. The forecast was too optimistic from 256 runs on the first eight problems and from both budgets on the further eight (within a factor two in only 68% of 96 cases over all sixteen problems), and its range is not an uncertainty interval; we do not claim that the forecast is validated. The basis flag raised no alarm on the further problems, so its usefulness rests on one batch of flags. We did not isolate marginal-likelihood fitting as the mechanism behind the Gaussian-process advantage, and the pilot diagnostic that we had hoped would predict the winner from 32 runs was not confirmed (Supplement, Sec. S3).

Limitations.

1. **Simulated problems.** All 44 problems are simulated; the sixteen problems solved by third-party code come from eight families (two levels each) of one finite-element library. The real-data check uses two datasets with scalar, noisy outputs, which violate assumption A2; it says that the tool behaves sensibly on them, not that it works for noisy field data.
2. **Design knowledge.** The pool, the kernel-ridge grids and the criteria were designed with knowledge of the 36 earlier problems, so the problems solved by third-party code are out-of-sample only in part, and they were chosen by us after the earlier results were known, so "independent" refers to the solver, not to the choice of problems. The eight further problems were run with the tool frozen and with criteria and predictions committed first, but they too were chosen by us, after the first results were known; the smoke tests of the run scripts used a problem that had already been seen. Smoke tests of the tool on three of the eight problems preceded the commit of the criteria; they showed test errors of some pool members and exposed a numerical bug in the leave-one-out computation, which was fixed before the evaluation runs (deviation V9). The text of the criteria was not changed afterwards, but its statement that no error had been seen was wrong when written.
3. **Few independent units.** Cells within a problem are not independent, so the intervals in table 4, which treat 24 cells as independent, are

optimistic. Seeds share the pool and test set and nest their training subsets.

4. **Forecast.** The forecast is a power-law extrapolation without guarantee; it was too low by more than a factor two in 7 of 48 cases, and the pre-specified criterion was met with a confidence interval that includes the threshold.
5. **Networks.** The default pool contains no network, the tool never recommends one, and nothing in its report warns that a network might do better. A network can be added to the pool as an option, which helped on one of eight further problems at several times the cost (section 6.7). The tuned network used 16 trials in a restricted space (depth 2–4, width 64–256) on one core; a variant with 48 trials, depth up to five and width up to 512 was better in most problems and changed the conclusion little, but a more heavily tuned or different architecture could do better.
6. **Cost.** Fifteen of the sixteen solvers take milliseconds per run, so on these problems the tool costs more than the simulations it replaces. One problem takes 1.52 seconds per run; there the surrogate was cheaper than the solver after about 335 predictions. Simulations that take minutes or hours were not tested, and that is where the case for the tool rests.
7. **Scope of the tool.** It assumes exchangeable runs on a common output grid, deterministic outputs, and a moderate number of inputs (only five of our 44 problems have ten or more).
8. **Literature.** Our search for work on automated surrogate selection and on learning-curve extrapolation was limited and mostly at abstract level, so related work may be incomplete.
9. **Process.** Hypotheses and code were committed before the runs they concern, but the history is our own, not an independent registry; several stages were designed after seeing earlier results (labelled post hoc); the independent reviews were performed by AI agents, not human reviewers.

Ethics and disclosure. No human or animal subjects were involved. The two real datasets are public (CC BY 4.0). The AI involvement is disclosed in the acknowledgments.

8. Conclusion and Future Scope

We built SurroSelect, a procedure and open tool that chooses a stand-in for a slow simulation by cross-validation, reports ties, checks whether the

POD basis limits accuracy and forecasts the error for more runs, and we tested it against criteria committed in advance on eight problems solved by an independent finite-element library, and again, with the tool frozen and predictions written down, on eight further ones. Selection was close to the oracle (the best pool member) in both batches (worst factor 5.2 over all 48 cells), although one fixed full-field Gaussian process does almost as well; the tuned network beat it by more than 25% in only six of 32 cells; the tool did not beat the benchmark's own selection among methods that include networks in either batch, and its error forecast was within a factor two in only 68% of cases over the sixteen problems and is not validated. Adding a tuned network to the pool helped where networks win, at several times the cost. On a solver taking 1.52 seconds per run, the surrogate paid for itself after about 335 predictions. A 44-problem study showed that the winner depends on smoothness, the number of inputs, the budget, the training effort of the networks and how widely a kernel method's regularisation is searched, and that the simple rule "reduced basis wins at large budgets" held on nine of the 16 problems solved by third-party code. Concrete next steps are (i) a rule for when to spend the cost of a tuned network in the pool; (ii) a forecast that models the error floor and gives a calibrated interval; (iii) tests on real field data from simulations that take minutes or hours, with the full cost of the simulations included; (iv) a public benchmark such as PDEBench [33]; and (v) independent human replication.

Acknowledgments

Use of AI tools. The study design, code, analyses, figures and the drafting of this paper were carried out with an AI system (Claude, Anthropic) acting as a research assistant under the authors' direction. The internal "reviews" and audits mentioned in the supplement were performed by AI agents, not by human reviewers. The authors are responsible for the content, have checked it (to be confirmed by the authors before submission), and all authors must approve the submission. [Funding, other acknowledgments and competing-interest statement: to be supplied by the authors.]

Data and code availability

All data are generated by the code in the project repository from problem names and seeds; raw outputs, aggregation scripts (which write every number in this paper) and figure scripts are included, and `code/run_all.py` reruns everything. The tool is the `surroselect` package (MIT licence; copyright

holder to be set by the authors). A permanent archive with a DOI has not been created yet: [authors to provide the public repository link or archive DOI before submission].

References

[1] A. Quarteroni, A. Manzoni, F. Negri, Reduced Basis Methods for Partial Differential Equations: An Introduction, UNITEXT, Springer, 2016. doi:10.1007/978-3-319-15431-2.

[2] J. S. Hesthaven, S. Ubbiali, Non-intrusive reduced order modeling of nonlinear problems using neural networks, Journal of Computational Physics 363 (2018) 55–78. doi:10.1016/j.jcp.2018.02.037.

[3] K. Bhattacharya, B. Hosseini, N. B. Kovachki, A. M. Stuart, Model reduction and neural networks for parametric PDEs, SMAI Journal of Computational Mathematics 7 (2021) 121–157. doi:10.5802/smai-jcm.74.

[4] M. Guo, J. S. Hesthaven, Reduced order modeling for nonlinear structural analysis using Gaussian process regression, Computer Methods in Applied Mechanics and Engineering 341 (2018) 807–826. doi:10.1016/j.cma.2018.07.017.

[5] S. Fresca, A. Manzoni, POD-DL-ROM: enhancing deep learning-based reduced order models for nonlinear parametrized PDEs by proper orthogonal decomposition, Computer Methods in Applied Mechanics and Engineering 388 (2022) 114181. doi:10.1016/j.cma.2021.114181.

[6] N. R. Franco, A. Manzoni, P. Zunino, A deep learning approach to reduced order modelling of parameter dependent partial differential equations, Mathematics of Computation 92 (340) (2023) 483–524. doi:10.1090/mcom/3781.

[7] N. McGreivy, A. Hakim, Weak baselines and reporting biases lead to overoptimism in machine learning for fluid-related partial differential equations, Nature Machine Intelligence 6 (10) (2024) 1256–1269. doi:10.1038/s42256-024-00897-5.

[8] L. Lu, P. Jin, G. Pang, Z. Zhang, G. E. Karniadakis, Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators, Nature Machine Intelligence 3 (2021) 218–229. doi:10.1038/s42256-021-00302-5.

[9] Z. Li, N. Kovachki, K. Azizzadenesheli, et al., Fourier neural operator for parametric partial differential equations, in: International Conference on Learning Representations, 2021, arXiv:2010.08895.

[10] P. Batlle, M. Darcy, B. Hosseini, H. Owhadi, Kernel methods are competitive for operator learning, Journal of Computational Physics 496 (2024) 112549. doi:10.1016/j.jcp.2023.112549.

[11] J. Westermann, B. Huber, T. O'Leary-Roseberry, J. Zech, Performance of neural and polynomial operator surrogates, arXiv:2604.00689 (2026).

[12] A. Bērziņš, J. Helmig, F. Key, S. Elgeti, Standardized non-intrusive reduced order modeling using different regression models with application to complex flow problems, arXiv:2006.13706 (2020).

[13] C. Mora, A. Yousefpour, S. Hosseinmardi, H. Owhadi, R. Bostanabad, Operator learning with Gaussian processes, Computer Methods in Applied Mechanics and Engineering 434 (2025) 117581. doi:10.1016/j.cma.2024.117581.

[14] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, A. Anandkumar, Neural operator: learning maps between function spaces with applications to PDEs, Journal of Machine Learning Research 24 (89) (2023) 1–97.

[15] K. Lee, K. T. Carlberg, Model reduction of dynamical systems on nonlinear manifolds using deep convolutional autoencoders, Journal of Computational Physics 404 (2020) 108973. doi:10.1016/j.jcp.2019.108973.

[16] A. Cohen, R. DeVore, Approximation of high-dimensional parametric PDEs, Acta Numerica 24 (2015) 1–159. doi:10.1017/s0962492915000033.

[17] C. Greif, K. Urban, Decay of the Kolmogorov N-width for wave problems, Applied Mathematics Letters 96 (2019) 216–222. doi:10.1016/j.aml.2019.05.013.

[18] C. E. Rasmussen, C. K. I. Williams, Gaussian Processes for Machine Learning, MIT Press, 2006. doi:10.7551/mitpress/3206.001.0001.

[19] K. Duraisamy, Predictivity and utility of neural surrogates of multiscale PDEs, arXiv:2604.20061 (2026).

[20] M. V. de Hoop, D. Z. Huang, E. Qian, A. M. Stuart, The cost-accuracy trade-off in operator learning with neural networks, Journal of Machine Learning 1 (3) (2022) 299–341. doi:10.4208/jml.220509.

[21] P. M. Wyder, J. Goldfeder, A. Yermakov, Y. Zhao, et al., Common task framework for a critical evaluation of scientific machine learning algorithms, arXiv:2510.23166 (2025).

[22] J. Bergstra, Y. Bengio, Random search for hyper-parameter optimization, Journal of Machine Learning Research 13 (2012) 281–305.

[23] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, A. Talwalkar, Hyperband: a novel bandit-based approach to hyperparameter optimization, Journal of Machine Learning Research 18 (2018) 1–52.

[24] T. Akiba, S. Sano, T. Yanase, T. Ohta, M. Koyama, Optuna: a next-generation hyperparameter optimization framework, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2019. doi:10.1145/3292500.3330701.

[25] J. Dodge, S. Gururangan, D. Card, R. Schwartz, N. A. Smith, Show your work: improved reporting of experimental results, in: Proceedings of EMNLP-IJCNLP 2019, 2019, pp. 2185–2194. doi:10.18653/v1/d19-1224.

[26] X. Bouthillier, P. Delaunay, M. Bronzi, et al., Accounting for variance in machine learning benchmarks, in: Proceedings of Machine Learning and Systems (MLSys), 2021, arXiv:2103.03098.

[27] B. Williams, S. Cremaschi, Surrogate model selection for design space approximation and surrogate-based optimization, Computer Aided Chemical Engineering 47 (2019) 353–358. doi:10.1016/B978-0-12-818597-1.50056-4.

[28] F. Mohr, J. N. van Rijn, Learning curves for decision making in supervised machine learning: a survey, Machine Learning 113 (2024) 8371–8425. doi:10.1007/s10994-024-06619-7.

[29] J. S. Rosenfeld, A. Rosenfeld, Y. Belinkov, N. Shavit, A constructive prediction of the generalization error across scales, in: International Conference on Learning Representations, 2020, arXiv:1909.12673.

[30] R. Milk, S. Rave, F. Schindler, pyMOR – generic algorithms and interfaces for model order reduction, SIAM Journal on Scientific Computing 38 (5) (2016) S194–S216. doi:10.1137/15M1026614.

[31] S. Marelli, B. Sudret, UQLab: a framework for uncertainty quantification in Matlab, in: Vulnerability, Uncertainty, and Risk (ICVRAM 2014), 2014. doi:10.1061/9780784413609.257.

[32] T. Gustafsson, G. D. McBain, scikit-fem: a Python package for finite element assembly, Journal of Open Source Software 5 (52) (2020) 2369. doi:10.21105/joss.02369.

[33] M. Takamoto, T. Praditia, R. Leiteritz, D. MacKinlay, F. Alesiani, D. Pflüger, M. Niepert, PDEBench: an extensive benchmark for scientific machine learning, NeurIPS 2022 Datasets and Benchmarks Track; arXiv:2210.07182 (2022).

[34] G. H. Golub, M. Heath, G. Wahba, Generalized cross-validation as a method for choosing a good ridge parameter, Technometrics 21 (2) (1979) 215–223. doi:10.1080/00401706.1979.10489751.

[35] C. R. Harris, K. J. Millman, S. J. van der Walt, et al., Array programming with NumPy, Nature 585 (2020) 357–362. doi:10.1038/s41586-020-2649-2.

[36] P. Virtanen, R. Gommers, T. E. Oliphant, et al., SciPy 1.0: fundamental algorithms for scientific computing in Python, Nature Methods 17 (2020) 261–272. doi:10.1038/s41592-019-0686-2.

[37] F. Pedregosa, G. Varoquaux, A. Gramfort, et al., Scikit-learn: machine learning in Python, Journal of Machine Learning Research 12 (2011) 2825–2830.

[38] A. Paszke, S. Gross, F. Massa, et al., PyTorch: an imperative style, high-performance deep learning library, in: Advances in Neural Information Processing Systems, Vol. 32, 2019, arXiv:1912.01703.

[39] E. B. Wilson, Probable inference, the law of succession, and statistical inference, Journal of the American Statistical Association 22 (158) (1927) 209–212. doi:10.1080/01621459.1927.10502953.

[40] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.